Faculty of Engineering & Technology

Department of Electrical and Computer Engineering

Computer Vision

ENCS5343

Final Project

Handwritten Arabic Numerals Recognition (0–9)

Prepared by:

Mohammad Abu Hijleh **ID: 1221350**

Instructor: **Dr. Dr. Aziz Qaroush**

Section: **1**

Date: **8/3/2026**

Table of Contents

Table of Contents I
Table of Figures I
List of Tables I
Introduction 1
Methodology and System Design 1
System Overview 1
Dataset Preparation 1
Baseline Model: K-Nearest Neighbors 2
Feature-Based Model: HOG + SVM 2
Deep Learning Model: Convolutional Neural Network 2
Data Augmentation 3
Experimental Results 3
Model Comparison 3
Per-Class Performance 4
Confusion Analysis 4
Discussion 5
Conclusion 5

Table of Figures

Figure 1: Model Comparison 3
Figure 2: Examples of the 16 misclassified test samples. 4
Figure 3: Confusion matrix for the CNN model with data augmentation. 4
Figure 4:Training and validation curves for the augmented CNN. 5

List of Tables

Table 1: Performance comparison of the implemented models. 3
Table 2: Per-class precision, recall, and F1-score for the CNN model with data augmentation. ... 4

Introduction

Handwritten digit recognition is a classical problem in computer vision and pattern recognition. Automatically identifying handwritten symbols has many practical applications, including bank check processing, postal mail sorting, form digitization, and document analysis. While extensive work has been conducted on Latin digits, recognizing handwritten Arabic numerals remains challenging due to variations in writing styles, stroke thickness, and digit shapes among writers.

The objective of this project is to design and implement a computer vision system capable of classifying handwritten Arabic numerals from images. The system receives an image containing a single handwritten digit and predicts the correct class among ten possible digits (0–9). To evaluate different approaches, both classical machine learning methods and deep learning models were explored.

The dataset used in this project contains **9,350 grayscale images** of handwritten Arabic-Indic digits collected from multiple writers. Each digit class contains **935 images**, and the images are approximately **28×28 pixels**. The dataset exhibits natural variation in handwriting style, orientation, and stroke structure.

Four models were implemented and compared:

- **K-Nearest Neighbors (KNN)** using raw pixel features
- **Support Vector Machine (SVM)** using Histogram of Oriented Gradients (HOG) features
- **Convolutional Neural Network (CNN)** trained on the original dataset
- **CNN with data augmentation** to improve generalization

All models were evaluated on a held-out test set using classification accuracy and additional metrics such as precision, recall, and F1-score. The results highlight the advantages of deep learning approaches for image recognition tasks compared with traditional feature-based methods.

Methodology and System Design

System Overview

The system follows a standard machine learning pipeline consisting of **data preparation, feature extraction, model training, and evaluation**. The workflow is illustrated as:

Dataset → Preprocessing → Feature Extraction → Model Training → Evaluation

Dataset Preparation

The dataset consists of **9,350 grayscale images** of handwritten Arabic numerals. Each class (0–9) contains **935 images**, stored in class-specific directories.

To train and evaluate the models, the dataset was split using **stratified sampling** to preserve class balance:

- **Training set:** 70% (6,545 images)

- **Validation set:** 15% (1,402 images)
- **Test set:** 15% (1,403 images)

The test set was reserved exclusively for final evaluation.

To avoid data leakage, normalization statistics were computed only on the training set. The calculated **mean (0.9811)** and **standard deviation (0.0763)** were then applied to the validation and test sets.

Baseline Model: K-Nearest Neighbors

The first approach uses a **K-Nearest Neighbors (KNN)** classifier with **k = 5**, trained directly on raw pixel intensities. Each image was flattened into a **784-dimensional vector** representing all pixel values.

During classification, the model assigns the label most common among the five nearest training samples. Although KNN is simple and easy to implement, it often performs poorly on image classification tasks because raw pixel values do not capture meaningful structural information and are sensitive to small spatial variations.

Feature-Based Model: HOG + SVM

To improve performance, **Histogram of Oriented Gradients (HOG)** features were extracted from each image. The configuration used **9 orientations**, **4×4 pixels per cell**, and **2×2 cells per block**, producing a **576-dimensional feature vector**.

These features were used to train a **Linear Support Vector Machine (SVM)** with regularization parameter **C = 10.0**. HOG descriptors capture edge orientations and local gradients, allowing the SVM to separate digit classes based on structural patterns rather than raw intensity values.

Deep Learning Model: Convolutional Neural Network

A custom **Convolutional Neural Network (CNN)** was implemented to automatically learn hierarchical image features. The network contains approximately **157,290 trainable parameters**, balancing model capacity with the relatively small dataset size.

The architecture consists of three convolutional blocks:

Block 1 (32 channels):
Two 3×3 convolution layers with BatchNorm and ReLU activation, followed by **2×2 max pooling** and spatial dropout (p = 0.25).

Block 2 (64 channels):
Two convolution layers with BatchNorm and ReLU, followed by pooling and dropout (p = 0.25).

Block 3 (128 channels):
A convolution layer followed by **Global Average Pooling (GAP)**, which reduces spatial dimensions to 1×1 and significantly lowers parameter count compared with traditional fully connected layers.

The classification head consists of a linear layer with **dropout (p = 0.5)** producing the final **10-class output**.

Training used the **Adam optimizer** (learning rate = 1×10^{-3}, weight decay = 1×10^{-4}) and **CrossEntropyLoss**. A **ReduceLROnPlateau** scheduler halved the learning rate when validation loss stagnated for five epochs, and **early stopping** halted training after ten epochs without improvement.

Data Augmentation

To improve generalization and reduce overfitting, data augmentation was applied during the second CNN training run. The following random transformations were used:

- Rotations up to **±12°**
- Small horizontal and vertical translations
- Scaling variations between **0.92 and 1.08**

Augmentation increases training data diversity and helps the model learn more robust representations that generalize better to unseen handwriting styles.

Experimental Results

Model Comparison

All models were evaluated using the same **test set of 1,403 images**.

Model	Test Accuracy	Macro F1
Raw Pixels + KNN (k=5)	68.64%	----------
HOG + SVM	90.59%	----------
CNN (no augmentation)	98.86%	0.9886
CNN (with augmentation)	98.86%	0.9886

Table 1: Performance comparison of the implemented models.

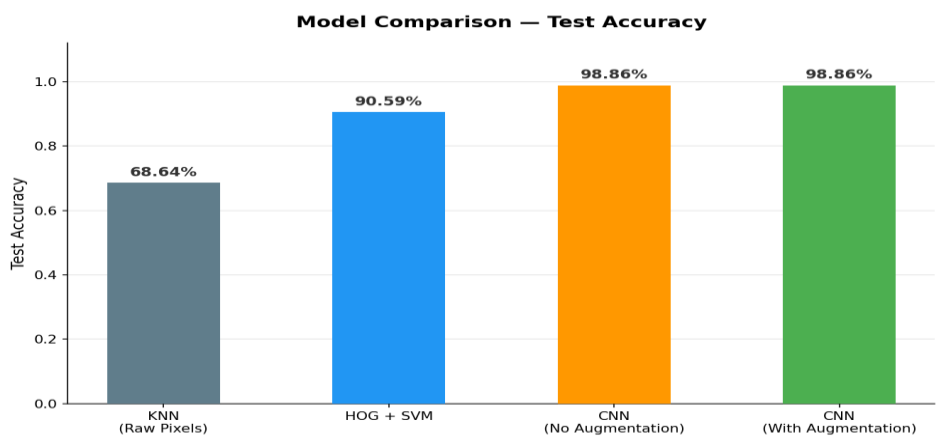

Figure 1: Model Comparison

The results show a clear improvement as more advanced methods are used. The KNN baseline achieved the lowest accuracy because raw pixel vectors contain limited discriminative information. Using HOG features significantly improved performance by capturing structural edge

patterns. The CNN models achieved the highest accuracy by automatically learning hierarchical features from the images.

Per-Class Performance

Digit	Precision	Recall	F1-Score
0	0.986	0.979	0.982
1	0.993	0.993	0.993
2	0.979	0.986	0.982
3	0.986	0.979	0.982
4	1.000	1.000	1.000
5	0.972	0.979	0.975
6	0.993	0.993	0.993
7	1.000	1.000	1.000
8	1.000	1.000	1.000
9	0.993	0.993	0.993

Table 2: Per-class precision, recall, and F1-score for the CNN model with data augmentation.

The augmented CNN achieved very high performance across all digit classes. Digits **4, 7, and 8** were classified with perfect precision and recall (F1 = 1.000). Overall, the model misclassified only **16 images** out of **1,403** in the test set.

Confusion Analysis

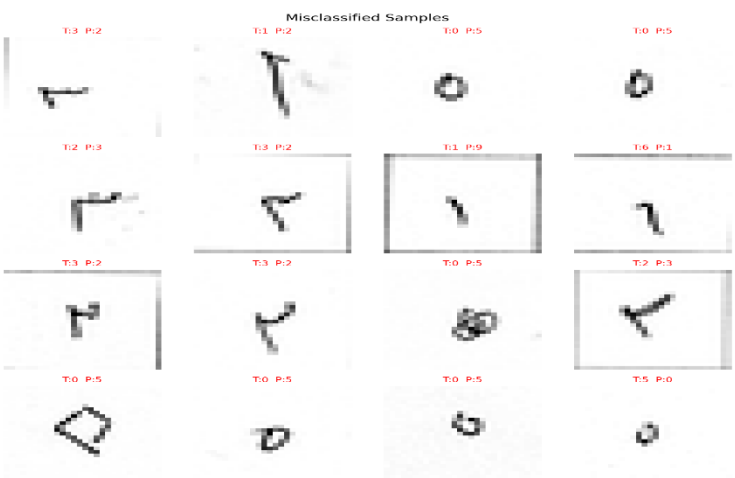

Figure 2: Examples of the 16 misclassified test samples.

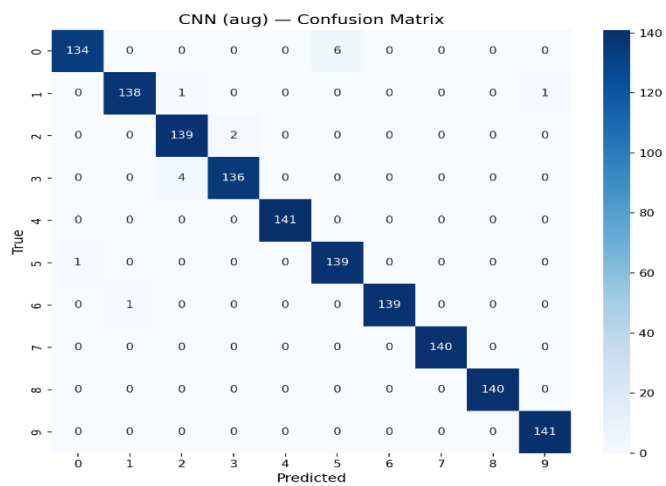

Figure 3: Confusion matrix for the CNN model with data augmentation.

Most digits were classified correctly, with only a small number of errors. The most frequent confusion occurred between digits **0 and 5 (6 cases)** because both may appear as circular shapes in handwritten form. Another source of error occurred between digits **2 and 3 (4 cases)** due to similar curved stroke patterns.

Discussion

The experimental results demonstrate that deep learning approaches significantly outperform traditional machine learning methods for handwritten digit recognition. The KNN model achieved the lowest performance because it relies on direct pixel comparisons without capturing structural information.

Using HOG features with an SVM classifier improved accuracy considerably, confirming the importance of feature extraction in image classification tasks. However, this approach still depends on manually designed features.

The CNN models achieved the highest accuracy because convolutional networks automatically learn hierarchical features such as edges, shapes, and higher-level patterns. The use of **Global Average Pooling** reduced the parameter count to **157,290** while maintaining strong performance.

Although both CNN models achieved identical test accuracy (98.86%), their training dynamics differed. Without augmentation, the model reached **99.42% training accuracy** but stopped early at epoch 41 due to overfitting signals. With augmentation, training accuracy reached **97.56%**, but the model trained steadily for **60 epochs** without overfitting. This indicates that augmentation acts as an effective regularizer by encouraging the network to learn more generalizable features.

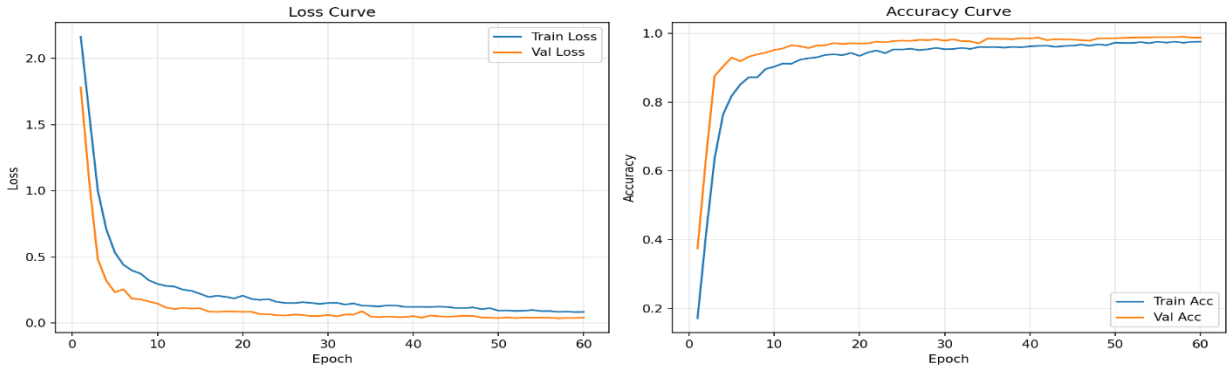

Figure 4:Training and validation curves for the augmented CNN.

Conclusion

In this project, a computer vision system for handwritten Arabic numeral recognition was developed and evaluated using both traditional machine learning and deep learning methods.

Experimental results demonstrate that convolutional neural networks significantly outperform classical approaches. The CNN model achieved **98.86% test accuracy**, misclassifying only **16 out of 1,403** images.

Data augmentation improved training stability and helped prevent overfitting by encouraging the model to learn more robust representations.

Future work could explore deeper architectures, targeted augmentation for confusing digit pairs, or larger datasets to further improve system robustness.